

TNPSC Combined Technical Services

Paper Code 554 — Unit II

DATA STRUCTURES AND ALGORITHMS

Expanded textbook-style study notes

A continuous, self-contained study text covering every topic in the Unit II syllabus. Each topic is explained from fundamentals through implementation ideas, operations, complexity and important distinctions.

PART I — LISTS, STACKS AND QUEUES

List ADT

An Abstract Data Type (ADT) specifies what operations a data structure supports and what those operations mean, without requiring a particular implementation. The List ADT represents an ordered finite sequence of elements. Typical operations include creating a list, testing whether it is empty, finding its size, retrieving an element, inserting an element, deleting an element, searching and traversing. The ADT distinction is important: a list can be implemented using an array, linked nodes, or another representation while presenting the same logical operations to its user. The choice of representation changes memory usage and operation costs. A list is ordered, so the position of an element matters. Duplicate values may be allowed unless a particular application imposes uniqueness. Common conceptual positions include the beginning, end and an index/position between elements.

- ADT describes behaviour, not implementation.
- Core operations: insertion, deletion, access, search and traversal.
- Array lists favour random access; linked lists favour insertion/deletion when a suitable node position is known.

Array-Based Implementation of Lists

An array-based list stores elements in contiguous memory. A fixed-capacity array can maintain a logical size smaller than its physical capacity. To insert at position i , elements from i onward generally have to be shifted one position to the right. To delete at position i , subsequent elements are shifted left. Random access is efficient because the address of element i can be calculated directly from the base address. This gives constant-time indexing. However, insertion and deletion near the beginning or middle are generally linear because of shifting. A dynamic array can grow when capacity is exhausted. A common strategy allocates a larger block, copies the elements and replaces the old storage. Although an individual resize costs $O(n)$, geometric growth such as doubling gives amortized $O(1)$ append time.

- Access by index: $O(1)$.
- Middle insertion/deletion: $O(n)$.
- Append: $O(1)$ amortized with geometric resizing.
- Contiguous storage provides good cache locality.

Linked List Implementation

A linked list represents a sequence using nodes connected by links rather than requiring contiguous storage. A node typically contains data and one or more pointers to other nodes. The list maintains a pointer to the first node and, depending on the design, may also maintain a tail pointer. Linked lists do not provide direct random access. To reach position i , traversal from the beginning is normally required, giving $O(n)$ access. Their major advantage is that insertion or deletion can be $O(1)$ once the relevant node or predecessor is already known, because links are changed rather than large blocks of elements being shifted. The cost of dynamic node allocation and pointer traversal means linked lists can have poorer cache locality than arrays.

- Nodes can be allocated independently.
- No shifting is needed for insertion/deletion.
- Random indexing is not efficient.
- Extra memory is required for link fields.

Singly Linked Lists

A singly linked list node contains data and a pointer to the next node. The last node's next pointer is null. Traversal proceeds only in the forward direction. Insertion at the front is $O(1)$: the new node points to the old head and becomes the head. Insertion after a known node is also $O(1)$. Deletion after a known predecessor is $O(1)$, because the predecessor's next pointer can bypass the removed node. Finding the predecessor by traversal may make the complete operation $O(n)$. Singly linked lists are simple and memory-efficient compared with doubly linked lists, but they cannot move directly backward.

- Head points to the first node.
- Last node points to null.
- Forward traversal only.
- Deleting a node generally requires access to its predecessor unless the design uses another technique.

Circularly Linked Lists

In a circular linked list, the last node points back to the first node rather than null. A circular list may be represented using a head pointer or, conveniently, a tail pointer whose next node is the head. Circular structure is useful when processing repeatedly wraps around, such as round-robin scheduling. Starting from any node, traversal can continue around the cycle, so termination must be based on returning to the starting point rather than reaching null. With a tail pointer, insertion at the front and back can both be performed efficiently because `tail.next` identifies the head.

- No null link at the logical end.
- Useful for cyclic or round-robin processing.
- Traversal must explicitly detect completion.
- Tail pointer can make end insertion efficient.

Doubly Linked Lists

A doubly linked list node contains data, a next pointer and a previous pointer. The first node's prev is null in a non-circular implementation, and the last node's next is null. The extra backward link allows traversal in both directions and makes deletion of a known node straightforward because its predecessor is available directly. Insertion before or after a known node can be performed by adjusting a constant number of links. The trade-off is additional memory per node and more pointer updates, which increase implementation complexity and the possibility of link-consistency errors.

- Supports forward and backward traversal.
- Deletion of a known node can be $O(1)$.
- Uses more memory than singly linked lists.
- Both next and prev links must remain consistent.

Applications of Lists

Lists are used whenever an ordered collection changes dynamically. Examples include maintaining symbol sequences, browser history, playlists, free lists in memory management, adjacency representations and intermediate results in algorithms. Linked lists are particularly useful where frequent insertion and deletion occur and random access is not central. Array-based lists are better where indexing and cache-friendly traversal dominate. Lists can also serve as building blocks for stacks, queues, hash-table chaining and graph adjacency structures.

- Choose representation according to the dominant operation.
- A list can model an ordered sequence without requiring unique keys.
- Linked structures are common in dynamic-memory applications.

Polynomial Manipulation

A polynomial can be represented as a collection of terms, where each term contains a coefficient and an exponent. For example, $5x^4 + 2x^2 + 7$ can be represented by pairs (5,4), (2,2), (7,0). A linked list is particularly useful because only nonzero terms need to be stored. Polynomial addition merges terms with equal exponents. If terms are kept sorted by exponent, the operation resembles merging two sorted lists. Polynomial multiplication generates products of terms and combines terms having equal resulting exponents. Representing sparse polynomials with lists avoids allocating storage for absent powers.

- Term = coefficient + exponent.
- Sorted exponent lists simplify addition.

- Like terms must be combined.
- Sparse representation stores only nonzero terms.

List Operations — Insertion, Deletion, Merge and Traversal

Insertion adds an element at a specified logical position. In an array list this may require shifting; in a linked list it usually requires link changes. Deletion removes an element and closes the resulting gap logically. Traversal visits elements in order and is fundamental for searching, printing, aggregation and transformation. A singly linked list is traversed by repeatedly following next pointers until null. A circular list requires a stopping condition based on returning to the starting node. Merging two lists means combining their contents. If the lists are sorted and the desired result is sorted, a two-pointer merge compares current elements and advances the pointer whose element is selected. This is the same central operation used in merge sort.

- Array insertion/deletion can require $O(n)$ shifts.
- Linked insertion/deletion is $O(1)$ once the exact link location is known.
- Traversal is $O(n)$.
- Sorted-list merge is $O(n+m)$.

Stack ADT

A stack is a linear ADT that follows Last In, First Out (LIFO). The principal operations are push, which inserts at the top; pop, which removes and returns the top item; and peek/top, which examines the top without removing it. isEmpty and size are common auxiliary operations. Stacks can be implemented using arrays or linked nodes. Array stacks offer compact storage and constant-time top operations, while linked stacks can grow dynamically until memory is exhausted. Stack overflow occurs when a bounded implementation has no space for another item. Stack underflow occurs when a pop is attempted on an empty stack.

- LIFO ordering.
- push/pop/peek are normally $O(1)$.
- Array implementation requires capacity management.
- Linked implementation uses one node link per element.

Evaluating Arithmetic Expressions

Stacks are fundamental for arithmetic-expression processing. Expressions may be written in infix notation, such as $A+B*C$, postfix notation, such as $ABC*+$, or prefix notation, such as $+A*BC$. Postfix evaluation is straightforward: scan left to right, push operands, and when an operator is encountered, pop the required operands, apply the operator and push the result. Parentheses are unnecessary in postfix because the order is encoded by the sequence. Converting infix to postfix commonly uses an operator stack. Operators are popped according to precedence and associativity. Parentheses force operators to remain on the stack until the matching opening parenthesis is processed.

- Postfix evaluation uses an operand stack.
- For a binary operator, the first popped value is the right operand.
- Operator precedence determines conversion order.
- Stacks also support balanced-parentheses checking.

Other Applications of Stacks

Stacks appear wherever the most recently created or visited item must be processed first. Function calls use a call stack to maintain activation records. Depth-first search can be implemented recursively or with an explicit stack. Backtracking algorithms use stacks to remember choices and states. Undo mechanisms can maintain a stack of previous states or actions. Browser navigation and expression parsing are other common examples. Compilers and interpreters use stack-based structures in parsing and execution models. The common pattern is reversal or nested dependency: the most recent unresolved item is handled before older ones.

- Function-call management.
- Depth-first traversal.
- Backtracking.
- Undo/redo systems.
- Syntax and expression parsing.

Queue ADT

A queue is a linear ADT that follows First In, First Out (FIFO). Elements are inserted at the rear and removed from the front. The fundamental operations are enqueue and dequeue, along with front/peek, isEmpty and size. Queues model waiting lines and scheduling systems. A correct implementation should make enqueue and dequeue efficient, typically $O(1)$. A simple array with front and rear indices can suffer wasted space after repeated dequeues unless elements are shifted or indices wrap around. Circular queues solve this by treating the storage as a ring.

- FIFO ordering.
- enqueue at rear; dequeue at front.
- front operation does not remove the item.
- Used extensively in breadth-first and scheduling algorithms.

Circular Queue Implementation

A circular queue uses a fixed array as a circular buffer. When the rear reaches the physical end, it wraps to index zero if space is available. Modular arithmetic is commonly used: $\text{next} = (\text{index} + 1) \% \text{capacity}$. Two common designs distinguish empty and full states using a count, a reserved slot, or separate state information. The exact full/empty condition depends on the chosen representation. Circular queues avoid shifting elements after dequeue and therefore support constant-time enqueue/dequeue under normal conditions.

- Wrap-around is implemented with modulo arithmetic.
- Enqueue/dequeue are $O(1)$.
- Full and empty conditions must be represented unambiguously.

Double Ended Queues (Deque)

A deque, pronounced 'deck', permits insertion and deletion at both ends. Operations commonly include push_front, push_back, pop_front and pop_back. A deque can be implemented using a circular array or a doubly linked list. It generalizes both stacks and queues: restricting operations to one end gives stack behaviour, while inserting at one end and deleting at the other gives queue behaviour. Deque implementations must efficiently maintain both front and rear positions.

- Both ends support insertion and deletion.
- Circular-array deque gives $O(1)$ end operations.
- Can model both stack-like and queue-like behaviour.

Priority Queues

A priority queue stores elements with associated priorities. Removal returns an element with the highest or lowest priority rather than necessarily the oldest inserted element. Ties may be resolved according to a secondary rule, such as insertion order, if required by the application. Binary heaps are a standard efficient implementation. In a min-priority queue, the smallest key is at the root; in a max-priority queue, the largest key is at the root. Insert and delete-min/delete-max take $O(\log n)$, while examining the extreme element is $O(1)$. Priority queues are central to Dijkstra's algorithm, event simulation, scheduling and graph algorithms.

- Priority determines removal order.
- Binary heap: insertion $O(\log n)$, delete-extreme $O(\log n)$, peek $O(1)$.

- Build-heap from n items can be $O(n)$.

Applications of Queues

Queues model processes in which requests are handled in arrival order or according to a controlled scheduling policy. Examples include CPU scheduling, printer queues, network packet buffering, breadth-first search, producer-consumer systems and event simulation. Variants such as priority queues and circular queues adapt the basic FIFO model to different requirements. In operating systems, a queue may contain ready processes; in networking, a queue may buffer packets awaiting transmission. The key abstraction is separation between producers adding work and consumers removing work according to a defined order.

- BFS uses a FIFO queue.
- Buffers absorb temporary rate differences between producers and consumers.
- Scheduling policies may use ordinary or priority queues.

PART II — TREES AND HEAPS

Binary Tree

A binary tree is a hierarchical structure in which each node has at most two children, conventionally called left and right. The root is the top node. A node with no children is a leaf. The depth of a node is its distance from the root; the height of a tree is the maximum depth of any node under a common convention. Binary trees can be full, complete, perfect, balanced or skewed. A full binary tree has either zero or two children at every node. A complete binary tree fills levels from left to right except possibly the last. A perfect binary tree has all internal nodes with two children and all leaves at the same depth. Binary trees are used in expression trees, search structures, heaps and hierarchical representations.

- A binary tree is not necessarily a binary search tree.
- Maximum nodes at depth d is 2^d .
- A complete binary tree is well suited to array-based heap representation.

Binary Search Tree

A binary search tree (BST) is a binary tree with an ordering invariant: keys in the left subtree are less than the node's key and keys in the right subtree are greater, subject to the duplicate-key policy used by the implementation. Searching compares the target with the current node and follows one subtree. Insertion follows the same decision path and attaches a new node at a suitable null position. Deletion has three major cases: deleting a leaf, deleting a node with one child, and deleting a node with two children. For two children, the node can be replaced by its inorder successor or predecessor. The efficiency depends on tree height. A balanced BST has $O(\log n)$ search/insert/delete, while a highly skewed BST can degrade to $O(n)$.

- Inorder traversal of a BST produces keys in sorted order.
- Two-child deletion often uses successor or predecessor.
- BST performance depends on height, not merely number of nodes.

Tree Traversals

Traversal systematically visits every node. Preorder visits root, left subtree, right subtree. Inorder visits left subtree, root, right subtree. Postorder visits left subtree, right subtree, root. Level-order visits nodes breadth-first, typically using a queue. Traversal order determines useful applications. Preorder can serialize a tree structure or process a node before its descendants. Inorder produces sorted order for a BST. Postorder is useful when children must be processed before a parent, such as deleting a tree. Level-order processes nodes by depth.

- Preorder: N-L-R.
- Inorder: L-N-R.
- Postorder: L-R-N.
- Level-order: breadth-first using a queue.

Tree Operations

Common tree operations include search, insertion, deletion, finding minimum/maximum, computing height, counting nodes and traversing. The exact complexity depends on the tree type. For an ordinary binary tree without ordering, searching may require visiting every node and is $O(n)$. In a BST, search follows one root-to-leaf path and is $O(h)$, where h is the height. Balanced trees keep h near $\log n$, while skewed trees may have h near n . Recursive definitions make tree algorithms natural: an operation on a tree can often be expressed in terms of operations on its left and right subtrees.

- Ordinary binary-tree search: $O(n)$.
- BST search: $O(h)$.
- Traversal of n nodes: $O(n)$.
- Height is the key structural measure for many tree complexities.

AVL Tree

An AVL tree is a self-balancing binary search tree. For every node, the height difference between its left and right subtrees, called the balance factor, is constrained to -1, 0 or +1. After insertion or deletion, an imbalance may appear. Rotations restore the height condition while preserving the BST ordering. The four common imbalance patterns are LL, RR, LR and RL. LL is corrected by a right rotation; RR by a left rotation; LR by left rotation of the child followed by right rotation; RL by right rotation of the child followed by left rotation. Because AVL trees maintain logarithmic height, search, insertion and deletion are $O(\log n)$.

- Balance factor = $\text{height}(\text{left}) - \text{height}(\text{right})$, under a common convention.
- Allowed balance factors: -1, 0, +1.
- Rotations preserve inorder key order.
- AVL trees are more strictly height-balanced than red-black trees.

Splay Tree

A splay tree is a self-adjusting binary search tree. After an access to a node, the node is moved toward the root using rotations called splaying. The main patterns are zig, zig-zig and zig-zag. A splay tree does not store an explicit balance factor or color. Individual operations can take $O(n)$, but a sequence of m operations has amortized $O(m \log n)$ complexity under the standard analysis, giving $O(\log n)$ amortized cost per operation. Splaying makes recently accessed nodes easier to access again, which can be advantageous when access patterns have locality.

- Self-adjusting rather than explicitly height-balanced.
- Zig: one rotation.
- Zig-zig: two rotations in the same direction.
- Zig-zag: two rotations in opposite directions.
- Amortized $O(\log n)$ per operation.

Red-Black Tree

A red-black tree is a self-balancing BST in which each node carries a red or black color and a set of invariants constrains the tree's height. Typical rules include that the root is black, red nodes cannot have red children, and every path from a node to descendant null leaves contains the same number of black nodes. Insertion and deletion may violate the invariants and are repaired using recoloring and rotations. These rules ensure height $O(\log n)$, so search, insertion and deletion are logarithmic. Red-black trees are widely used for ordered associative containers because they provide predictable logarithmic worst-case performance while requiring less strict balancing than AVL trees.

- Red-black invariants bound tree height.
- Rebalancing uses rotations and recoloring.
- Search/insert/delete: $O(\log n)$ worst case.

Binary Heap

A binary heap is a complete binary tree satisfying a heap-order property. In a min-heap, every parent key is less than or equal to its children; in a max-heap, every parent key is greater than or equal to its children. Because the tree is complete, it is commonly represented in an array without explicit pointers. With zero-based indexing, the children of index i are commonly $2i+1$ and $2i+2$, while the parent is $\text{floor}((i-1)/2)$ for $i > 0$. Insertion adds an element at the end and repeatedly swaps it upward until the heap property is restored. Delete-min/delete-max replaces the root with the final element and moves it downward. Building a heap from an arbitrary array can be done in $O(n)$ using bottom-up heapify.

- Complete-tree shape + heap-order property.
- Peek extreme: $O(1)$.
- Insert/delete extreme: $O(\log n)$.
- Build heap: $O(n)$.

Skew Heap

A skew heap is a self-adjusting heap supporting efficient merge operations without explicit balance information. Its defining operation is merge. During merging, the roots are compared, and the appropriate root remains while subheaps are recursively combined; children are swapped as part of the restructuring. Skew heaps are attractive when melding two priority queues is frequent. Their operations have good amortized bounds rather than strict worst-case logarithmic guarantees for every individual operation.

- Main strength: efficient meld/merge.
- No explicit rank or balance field.
- Self-adjusting structure.
- Insert and delete can be expressed using merge.

Leftist Heap

A leftist heap is a heap-ordered binary tree designed to make merging efficient. Each node stores a null-path length (or equivalent rank), and the leftist property requires the rank of the left child to be at least the rank of the right child. Merge compares roots, recursively merges the appropriate right subtree and then swaps children if needed to maintain the leftist property. Because the right spine is short, merge takes $O(\log n)$. Insert and delete-min can be implemented using merge and therefore are $O(\log n)$. The key difference from a skew heap is that a leftist heap explicitly maintains a structural rank property.

- Heap order + leftist rank property.
- Merge: $O(\log n)$.
- Insert/delete-min: $O(\log n)$.
- Right spine remains logarithmic.

Binomial Heap

A binomial heap is a collection of binomial trees satisfying heap order, with at most one tree of each degree. A binomial tree B_k contains 2^k nodes and has height k . The structure supports efficient melding by merging root lists by degree and linking trees of equal degree. This is analogous to binary addition: when two trees of the same degree meet, one becomes the child of the other, producing a tree of the next degree. Binomial heaps support merge in $O(\log n)$, insertion in $O(\log n)$ or better depending on implementation, and minimum extraction in $O(\log n)$.

- At most one binomial tree of each degree.
- B_k has 2^k nodes.
- Meld resembles binary carrying.
- Useful for merge-heavy priority queues.

Fibonacci Heap

A Fibonacci heap is a collection of heap-ordered trees designed to make several priority-queue operations efficient in amortized analysis. It maintains a set of root trees and delays consolidation. Insert, find-min and decrease-key have especially strong amortized bounds: insert and find-min are $O(1)$, while decrease-key is $O(1)$ amortized. Extract-min is $O(\log n)$ amortized. Melding two heaps can be $O(1)$ amortized. Fibonacci heaps are theoretically important for graph algorithms because algorithms that perform many decrease-key operations can benefit from the improved amortized complexity. The implementation is substantially more complex than a binary heap.

- Insert: $O(1)$ amortized.
- Find-min: $O(1)$.
- Decrease-key: $O(1)$ amortized.
- Extract-min: $O(\log n)$ amortized.
- Meld: $O(1)$ amortized.

PART III — SORTING, SEARCHING AND HASHING

Insertion Sort

Insertion sort builds a sorted portion of the array one element at a time. For each new element, it shifts larger elements in the sorted portion to the right and inserts the element into the resulting position. It is simple, stable and in-place. Its worst-case and average running time are $O(n^2)$, while its best case is $O(n)$ when the input is already sorted. It performs particularly well on small arrays and nearly sorted data.

- Worst/average: $O(n^2)$.
- Best: $O(n)$.
- Stable and in-place in its standard form.
- Adaptive to nearly sorted input.

Selection Sort

Selection sort repeatedly selects the smallest remaining element and places it at the beginning of the unsorted region. After the first pass, the smallest element is fixed; after the second, the next smallest is fixed, and so on. The algorithm performs $O(n^2)$ comparisons regardless of input order. Standard selection sort performs relatively few swaps, which can be useful when writes are expensive. The common basic version is not stable unless modified.

- Time: $O(n^2)$ in best, average and worst cases.
- Simple and in-place.
- Usually not stable.
- Uses $O(1)$ auxiliary space.

Shell Sort

Shell sort generalizes insertion sort by first comparing elements separated by a gap. It performs insertion-sort-like passes for decreasing gap values, ending with gap 1. Early passes move elements long distances, reducing disorder before the final insertion-sort pass. Its complexity depends strongly on the chosen gap sequence. Unlike elementary $O(n^2)$ sorts, good gap sequences can provide substantially better practical performance, but there is no single universal complexity independent of the sequence.

- Gap sequence determines performance.
- Final gap is normally 1.
- In-place.
- Usually unstable.

Bubble Sort

Bubble sort repeatedly compares adjacent elements and swaps them when they are out of order. After a full pass, the largest unsorted element has 'bubbled' toward the end. The basic algorithm is $O(n^2)$ in average and worst cases. With a swapped flag, an already sorted array can be recognized in $O(n)$. Bubble sort is stable when only out-of-order adjacent elements are swapped.

- Best with early-exit optimization: $O(n)$.
- Average/worst: $O(n^2)$.
- Stable in standard form.
- In-place.

Quick Sort

Quicksort selects a pivot and partitions the array so that elements are placed on appropriate sides of the pivot. It recursively sorts the partitions. Average expected performance is $O(n \log n)$ with suitable pivot selection, while the worst case is $O(n^2)$, such as repeatedly choosing a poor pivot. Randomization or good pivot strategies reduce the chance of pathological partitions. Quicksort is typically in-place apart from recursion stack and is not stable in its standard form.

- Partitioning is the central operation.
- Average/expected: $O(n \log n)$.
- Worst: $O(n^2)$.
- Recursion depth depends on partition quality.
- Usually not stable.

Merge Sort

Merge sort divides the sequence into halves, recursively sorts each half, and merges the sorted halves. The merge step takes linear time in the number of elements being merged. Its running time is $O(n \log n)$ in best, average and worst cases. Standard array merge sort requires $O(n)$ auxiliary storage. It is stable when the merge procedure preserves the order of equal keys. Merge sort is naturally suited to linked lists and external sorting because merging sequential runs is efficient.

- Time: $O(n \log n)$ in all cases.
- Stable in standard stable implementation.
- Array version usually uses $O(n)$ auxiliary space.
- Merge operation: $O(n)$.

Radix Sort

Radix sort sorts keys digit by digit rather than comparing complete keys. In least-significant-digit (LSD) radix sort, stable sorting is performed on each digit from least significant to most significant. Most-significant-digit (MSD) radix sort starts from the highest digit and recursively partitions. Its complexity is often expressed as $O(d(n+k))$, where d is the number of digit positions and k is the radix/base, assuming the digit-level stable sort is linear. Radix sort is useful when keys have suitable bounded representations and digit extraction is efficient.

- Non-comparison sorting technique.
- Requires stable digit sorting for LSD radix sort.
- Complexity depends on number of digits and radix.
- Useful for fixed-width integers and strings under suitable conditions.

Heap Sort

Heap sort first builds a max-heap for ascending order, then repeatedly places the maximum at the end and restores the heap property on the remaining prefix. Building the heap is $O(n)$, and each of n removals takes $O(\log n)$, giving $O(n \log n)$ overall. Heap sort has $O(n \log n)$ worst-case time and $O(1)$ auxiliary array space in its standard in-place form. It is not stable. Its predictable worst-case bound makes it useful when guaranteed logarithmic sorting time is required.

- Build heap: $O(n)$.
- Total: $O(n \log n)$.
- Worst-case: $O(n \log n)$.
- In-place, but not stable.

Linear Search

Linear search examines elements sequentially until the target is found or the collection ends. It requires no sorted order and works with any sequence that permits sequential inspection. For n elements, the worst case requires n comparisons and is $O(n)$. The best case is $O(1)$ when the target is first. Linear search is appropriate for small or unsorted collections and for structures where random access is unavailable.

- No sorting prerequisite.
- Worst: $O(n)$.
- Best: $O(1)$.
- Works naturally on linked lists.

Binary Search

Binary search requires a sorted sequence and repeatedly compares the target with the middle element. If the target is smaller, the search continues in the left half; if larger, in the right half. Each comparison approximately halves the remaining search interval. The running time is $O(\log n)$ for random-access arrays. An iterative implementation uses $O(1)$ auxiliary space. Applying binary search directly to a linked list does not provide the same benefit because finding the middle repeatedly is not constant-time.

- Requires sorted data.
- Time on random-access array: $O(\log n)$.
- Each step halves the search interval.
- Use careful midpoint calculation to avoid integer overflow in languages where that matters.

Hash Functions

Hashing maps a key to an index or bucket using a hash function. A good hash function distributes keys uniformly across the table so that collisions are infrequent. The table size and hash function determine the mapping. A common simple family for integer keys uses modulo a table size, but the choice of table size and key transformation affects distribution. For strings, hashes combine character values using a mixing process. A collision occurs when different keys map to the same table position. Collision resolution is therefore a fundamental part of hash-table design.

- Hash function should be deterministic for a given key/table configuration.
- Uniform distribution reduces expected collisions.
- Collision means distinct keys share a hash location.
- Load factor α = number of stored keys / table capacity.

Separate Chaining

Separate chaining resolves collisions by maintaining a collection of entries at each table bucket, commonly a linked list. When several keys hash to the same index, they are stored in that bucket's chain. Expected lookup, insertion and deletion are approximately $O(1)$ under a good hash function and controlled load factor, but worst-case lookup can become $O(n)$ if all keys collide. Chaining can support load factors above 1 because each bucket can contain multiple elements.

- Each bucket stores a chain/container of entries.
- Load factor can exceed 1.
- Expected operations are $O(1)$ under good distribution.
- Worst case $O(n)$.

Open Addressing

Open addressing stores every key directly in the hash table. On collision, a probing sequence searches for another slot. The main methods are linear probing, quadratic probing and double hashing. Linear probing examines consecutive positions and has good cache locality but can create primary clustering. Quadratic probing uses increasing quadratic offsets and reduces some clustering. Double hashing uses a second hash function to determine the probe step and can provide better distribution. Deletion is more complicated than insertion because simply clearing a slot can break a future probe sequence. A tombstone marker is often used to indicate a deleted slot that probing must pass through.

- All entries remain inside the table.

- Load factor must remain below 1.
- Linear probing can cause primary clustering.
- Deletion often uses tombstones.
- Performance degrades as the load factor approaches 1.

Rehashing

Rehashing occurs when a hash table is resized and existing keys are inserted into a new table using the new capacity and hash mapping. It is necessary because changing the table size usually changes the index computed by the hash function. A typical strategy grows the table when its load factor crosses a threshold. The old entries are traversed and inserted into the new table. The resize operation itself costs $O(n)$, but with geometric growth the cost can be amortized across many insertions.

- Resize changes bucket/index mapping.
- Existing entries must be reinserted.
- One rehash is $O(n)$.
- Geometric resizing provides amortized efficient insertion.

Extendible Hashing

Extendible hashing is a dynamic hashing technique designed especially for external storage. It uses a directory of pointers to buckets. A global depth describes how many hash bits index the directory, while each bucket has a local depth describing how many bits distinguish that bucket. When a bucket overflows, it can be split. If its local depth reaches the global depth, the directory can be doubled before the split. This allows the structure to grow incrementally rather than rebuilding an entire table after every capacity increase. The technique is useful in database and file systems because buckets correspond naturally to storage blocks.

- Directory uses hash-bit prefixes.
- Global depth controls directory size.
- Local depth controls a bucket's split granularity.
- Bucket splits are incremental.

PART IV — GRAPH ALGORITHMS

Graphs — Basic Representation

A graph consists of vertices (nodes) and edges connecting pairs of vertices. Graphs may be directed or undirected, weighted or unweighted. In a directed graph, an edge has an orientation from source to destination. In an undirected graph, the connection is symmetric. Common representations are adjacency matrices and adjacency lists. An adjacency matrix uses $O(V^2)$ storage and provides $O(1)$ edge-existence lookup, while an adjacency list uses $O(V+E)$ storage and is efficient for sparse graphs. Graph algorithms often depend on whether edges are weighted, whether negative weights exist and whether the graph contains cycles.

- V = number of vertices; E = number of edges.
- Adjacency matrix: $O(V^2)$ space.
- Adjacency list: $O(V+E)$ space.
- Sparse graphs generally favour adjacency lists.

Graph Traversals

Breadth-first search (BFS) explores vertices level by level using a queue. Starting from a source, it discovers all unvisited neighbours, then their neighbours, and so on. In an unweighted graph, BFS computes shortest path distances measured in number of edges. Depth-first search (DFS) follows a path as deeply as possible before backtracking. It can be implemented recursively or with an explicit stack. DFS is useful for cycle detection, connected components, topological ordering, strongly connected component algorithms and many structural analyses. With adjacency lists, BFS and DFS run in $O(V+E)$ because each vertex and edge is processed a bounded number of times.

- BFS $\rightarrow$ queue.
- DFS $\rightarrow$ recursion/stack.
- BFS gives shortest unweighted path distances.
- Adjacency-list traversal: $O(V+E)$.

Minimum Spanning Tree

For a connected, weighted, undirected graph, a spanning tree connects all vertices with exactly $V-1$ edges and no cycles. A minimum spanning tree (MST) is a spanning tree with minimum total edge weight. Kruskal's algorithm sorts edges by increasing weight and adds an edge if it connects two previously separate components. A disjoint-set union (union-find) structure efficiently detects whether adding the edge would create a cycle. Prim's algorithm grows one tree from a starting vertex, repeatedly adding the minimum-weight edge crossing from the tree to an unvisited vertex. MST algorithms assume the appropriate undirected weighted setting. They differ from shortest-path algorithms: an MST minimizes total tree weight, not the distance from a particular source.

- MST has $V-1$ edges.
- Kruskal: sort edges + union-find.
- Prim: grow a single tree using minimum crossing edge.
- MST is not the same as shortest-path tree.

Shortest Path Algorithms

Shortest-path problems seek minimum-weight paths between vertices. Single-source shortest path algorithms include BFS for unweighted graphs, Dijkstra for graphs with nonnegative edge weights, and Bellman-Ford when negative edge weights may exist. Dijkstra repeatedly finalizes the unsettled vertex with the smallest tentative distance and relaxes outgoing edges. With a binary heap and adjacency list, a common bound is $O((V+E) \log V)$, often written $O(E \log V)$ for connected sparse graphs under suitable assumptions. Negative-weight edges invalidate Dijkstra's greedy correctness because a vertex thought to be finalized can later receive a shorter path. Bellman-Ford handles negative edges and can detect reachable negative cycles.

- BFS → unweighted shortest paths.
- Dijkstra → no negative edge weights.
- Bellman-Ford → permits negative edges and detects negative cycles reachable from source.
- Relaxation is the core shortest-path operation.

Directed Acyclic Graph (DAG)

A directed acyclic graph is a directed graph containing no directed cycle. DAGs naturally represent dependencies, precedence relationships and workflows. Every finite DAG has at least one vertex with indegree zero and at least one vertex with outdegree zero. This property enables topological ordering. DAGs also allow efficient shortest- or longest-path dynamic programming when vertices are processed in topological order.

- No directed cycles.
- Dependency graphs are often DAGs.
- Topological order exists iff the directed graph is acyclic.

Topological Ordering

A topological ordering is a linear ordering of the vertices of a DAG such that for every directed edge $u \rightarrow v$, u appears before v . It can be computed using Kahn's algorithm or DFS. Kahn's algorithm maintains indegrees. Vertices with indegree zero are placed in a queue; removing one conceptually removes its outgoing edges, decreasing neighbour indegrees. If fewer than V vertices are processed, a directed cycle exists. DFS-based ordering places a vertex after all descendants have been explored, then reverses the resulting finishing sequence.

- Only DAGs have topological orderings.
- Kahn's algorithm uses indegrees.
- Time with adjacency lists: $O(V+E)$.
- Cycle detection occurs if not all vertices can be ordered.

All-Pairs Shortest Path Algorithms

The all-pairs shortest-path problem asks for shortest distances between every ordered pair of vertices. One approach is to run a suitable single-source algorithm from each source. Another is dynamic programming using the Floyd-Warshall algorithm. For sparse graphs with suitable weights, repeated Dijkstra may be effective when there are no negative edges. Repeated Bellman-Ford handles negative edges but can be expensive. Floyd-Warshall is simple and systematic and works well for dense graphs and moderate vertex counts. The choice depends on graph density, presence of negative weights, required output and graph size.

- All-pairs output contains $V \times V$ distances.
- Floyd-Warshall: $O(V^3)$.
- Repeated single-source methods may be better for sparse graphs.

Floyd-Warshall Algorithm

Floyd-Warshall is a dynamic-programming algorithm for all-pairs shortest paths. Let $D^k(i,j)$ represent the shortest path from i to j whose internal vertices come from the first k allowed vertices. For each intermediate vertex k , the algorithm tests whether travelling $i \rightarrow k \rightarrow j$ is shorter than the current $i \rightarrow j$ path: $D[i][j] = \min(D[i][j], D[i][k] + D[k][j])$. After all k are processed, the matrix contains shortest distances. The algorithm runs in $O(V^3)$ time and $O(V^2)$ space. Negative edge weights are allowed, but a negative cycle makes shortest-path distances undefined for affected pairs. A negative diagonal entry after processing indicates a negative cycle.

- Dynamic programming over allowed intermediate vertices.
- Time: $O(V^3)$.
- Space: $O(V^2)$.

- Negative edges allowed; negative cycles require special interpretation.

Bellman-Ford Algorithm

Bellman-Ford computes single-source shortest paths in graphs that may contain negative-weight edges. It repeatedly relaxes every edge. After $V-1$ complete passes, every shortest simple path has been accounted for because a simple path has at most $V-1$ edges. A further pass that can still relax an edge reachable from the source indicates a reachable negative-weight cycle. Such a cycle means there is no finite shortest distance for vertices that can be reached from the cycle. The standard complexity is $O(VE)$, which is slower than Dijkstra on nonnegative-weight graphs but more general with respect to edge weights.

- Supports negative edge weights.
- Detects reachable negative cycles.
- Standard time: $O(VE)$.
- Uses repeated edge relaxation.

Network Flow Algorithms

A flow network is a directed graph with capacities on edges, a source s and a sink t . A feasible flow respects capacity constraints and flow conservation at intermediate vertices. The value of a flow is the total amount leaving the source (equivalently entering the sink under conservation). The residual network represents how the current flow can be increased or rerouted. An augmenting path is a path from source to sink in the residual graph along which additional flow can be sent. The maximum-flow problem seeks the greatest possible flow value. The max-flow/min-cut theorem states that maximum flow value equals the minimum capacity of an s - t cut.

- Capacity constraint: $0 \leq f(u,v) \leq c(u,v)$.
- Flow conservation at intermediate vertices.
- Residual capacity enables augmentation and rerouting.
- Maximum flow equals minimum cut capacity.

Ford-Fulkerson Algorithm

Ford-Fulkerson starts with zero flow and repeatedly finds an augmenting path in the residual network. It sends the maximum possible additional flow along that path, called the bottleneck residual capacity, then updates forward and reverse residual edges. When no augmenting path remains, the resulting flow is maximum under the standard assumptions. If capacities are integers, each augmentation increases flow by at least one unit, giving a finite termination argument, though the number of augmentations can be large. The Edmonds-Karp implementation chooses augmenting paths using BFS and has a polynomial $O(VE^2)$ bound. General Ford-Fulkerson's complexity depends on path selection and capacity assumptions.

- Augment by bottleneck capacity.
- Reverse residual edges permit undoing/re-routing previous choices.
- No augmenting path $\rightarrow$ maximum flow.
- Edmonds-Karp uses BFS and gives $O(VE^2)$.

PART V — ALGORITHM ANALYSIS AND DESIGN TECHNIQUES

Amortized Analysis of Algorithms

Amortized analysis studies the average cost per operation over a sequence of operations, without relying on probability. The goal is to show that although some individual operations may be expensive, the total cost of a long sequence is bounded by a manageable aggregate. Three standard techniques are aggregate analysis, the accounting method and the potential method. In aggregate analysis, the total cost of n operations is bounded directly. In the accounting method, operations are assigned amortized charges and saved credit pays for future expensive operations. The potential method defines a potential function representing stored work. Dynamic-array append is a classic example: most appends are cheap, but resizing is expensive. With geometric capacity growth, the amortized append cost is $O(1)$.

- Amortized $\neq$ average-case probability analysis.
- Expensive operations are paid for by cheaper operations in the sequence.
- Dynamic arrays and splay trees are standard examples.

Asymptotic Analysis

Asymptotic analysis describes how resource requirements grow as input size n becomes large. Time complexity measures operations or running time; space complexity measures additional memory requirements. Big- O gives an asymptotic upper bound, Big- Ω gives a lower bound and Big- Θ gives a tight asymptotic bound when both upper and lower bounds match. Constants and lower-order terms are ignored because growth dominates for sufficiently large n . Common growth rates include $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(n^3)$ and exponential $O(2^n)$. The difference between these rates becomes increasingly significant as input size grows.

- $O(g(n))$ means bounded above asymptotically by a constant multiple of $g(n)$.
- $\Theta(g(n))$ expresses a tight asymptotic bound.
- Nested loops often multiply their iteration counts.
- Sequential blocks usually contribute additively, with the dominant term determining the final order.

Solving Recurrence Equations

Recurrence equations describe the running time of recursive algorithms in terms of smaller inputs. For example, merge sort satisfies $T(n) = 2T(n/2) + O(n)$: two recursive subproblems of half size plus linear merge work. Common techniques include expansion/substitution, recursion trees and the Master Theorem for recurrences of the form $T(n) = aT(n/b) + f(n)$. The Master Theorem compares $f(n)$ with $n^{\log_b a}$. In the standard cases, if f is polynomially smaller, T is dominated by recursive work; if f matches it, an additional logarithmic factor appears; if f is polynomially larger and a regularity condition holds, T is dominated by f .

- Binary search: $T(n) = T(n/2) + O(1) \rightarrow O(\log n)$.
- Merge sort: $2T(n/2) + O(n) \rightarrow O(n \log n)$.
- Simple linear recursion may yield $T(n) = T(n-1) + O(1) \rightarrow O(n)$.
- Always identify number of subproblems, their size and non-recursive work.

Greedy Algorithms

A greedy algorithm constructs a solution step by step, making the locally best choice according to a specified rule and generally never revisiting that choice. Greedy methods can be optimal when the problem has the appropriate structural properties, commonly described through greedy-choice and optimal-substructure properties. Examples include Kruskal's and Prim's minimum spanning tree algorithms, Dijkstra under nonnegative edge weights, activity selection and Huffman coding. A greedy strategy is not automatically correct merely because each choice looks beneficial. Correctness normally requires a proof, often an exchange argument showing that an optimal solution can be transformed to contain the greedy choice without becoming worse.

- Greedy makes an immediate local choice.

- Correctness requires structural justification.
- MST algorithms are important greedy examples.
- Greedy is generally simpler than dynamic programming when applicable.

Dynamic Programming

Dynamic programming (DP) solves problems with overlapping subproblems and optimal substructure by storing results of smaller subproblems so they are not recomputed. Top-down DP uses recursion with memoization. Bottom-up DP fills a table iteratively from base cases toward the desired state. State design is central: the state must contain enough information to determine future decisions without retaining unnecessary history. Classic examples include Fibonacci optimization, 0/1 knapsack, matrix-chain multiplication, longest common subsequence and many shortest-path formulations. DP often converts exponential recursive solutions into polynomial-time algorithms when the number of states and transitions is manageable.

- Memoization = top-down caching.
- Tabulation = bottom-up computation.
- State count $\times$ transition cost often determines complexity.
- Overlapping subproblems distinguish DP from simple divide-and-conquer.

Divide and Conquer

Divide and conquer solves a problem by dividing it into smaller independent subproblems, recursively solving them and combining their results. The three stages are divide, conquer and combine. Merge sort is a classic example: divide the array into halves, recursively sort each half and merge. Quicksort divides by partitioning around a pivot and then recursively solves the partitions, with little explicit combine work. The subproblems are typically independent, unlike dynamic programming where overlapping subproblems make memoization valuable. Recurrence equations naturally model divide-and-conquer running time.

- Divide $\rightarrow$ create smaller subproblems.
- Conquer $\rightarrow$ solve recursively.
- Combine $\rightarrow$ construct the final solution.
- Merge sort and quicksort are classic examples.

COMPLEXITY REFERENCE TABLE

The following table summarizes the principal asymptotic costs discussed in this unit. Actual complexity can depend on implementation details, balance conditions, hashing quality and the exact operation being measured.

- **Array-list access** — $O(1)$
- **Linked-list access by position** — $O(n)$
- **Linked-list insertion/deletion at known position** — $O(1)$
- **Stack push/pop/peek** — $O(1)$
- **Queue enqueue/dequeue** — $O(1)$
- **BST search/insert/delete** — $O(h)$; $O(\log n)$ when balanced, $O(n)$ worst-case
- **AVL / Red-black search, insert, delete** — $O(\log n)$
- **Binary heap insert/delete-extreme** — $O(\log n)$
- **Binary heap build** — $O(n)$
- **Insertion sort** — $O(n^2)$ worst; $O(n)$ best
- **Selection sort** — $O(n^2)$
- **Bubble sort** — $O(n^2)$ worst; $O(n)$ best with early exit
- **Quicksort** — $O(n \log n)$ average/expected; $O(n^2)$ worst
- **Merge sort** — $O(n \log n)$
- **Heap sort** — $O(n \log n)$
- **Linear search** — $O(n)$
- **Binary search** — $O(\log n)$
- **Hash lookup** — $O(1)$ expected; $O(n)$ worst
- **BFS / DFS with adjacency list** — $O(V+E)$
- **Floyd-Warshall** — $O(V^3)$
- **Bellman-Ford** — $O(VE)$
- **Edmonds-Karp** — $O(VE^2)$

CORE ALGORITHM PATTERNS

```
/* Binary search – iterative form */
int low = 0, high = n - 1;
while (low <= high) {
    int mid = low + (high - low) / 2;
    if (a[mid] == key) return mid;
    if (a[mid] < key) low = mid + 1;
    else high = mid - 1;
}
return -1;

/* Singly linked-list traversal */
Node *p = head;
while (p != NULL) {
    process(p->data);
    p = p->next;
}

/* BFS conceptual pattern */
enqueue(source);
mark(source);
while (!empty()) {
    u = dequeue();
    for each neighbour v of u:
        if v is unvisited:
            mark(v);
            enqueue(v);
}

/* Dynamic programming pattern */
dp[base_state] = base_value;
for each state in dependency order:
    dp[state] = best/computed value from earlier states;
```

These patterns are included to connect the theoretical explanations with the standard implementations. The central ideas are the invariants: ordered-search structure, link consistency, FIFO/LIFO discipline, heap order, relaxation, and reuse of computed subproblems.